mb-free___transport-rendezvous-2048

An AgentMPI run analysis

Abstract

This is the last cell of a ten-cell transport sweep at which both modes still run. Eight agent-free ranks ring-exchange a 2,048-word payload under `Mode.RENDEZVOUS`, delivering an envelope rather than the text; the run completed in 0.02 s over 34 events with zero admissions, zero context high water on every rank, and 21,560 tokens deferred. Its eager twin also completed, admitting 2,695 tokens into each receiver — 8.2% of the 32,768-token context budget but 65.8% of the 4,096-token unexpected-message credit that gates the eager path. That makes this the last place in the sweep where the two modes can be compared on equal terms, and the comparison is unflattering to the intuitive reading of the trade. The two runs write the same blob, send the same eight envelopes, and finish in indistinguishable time. Nothing physical is saved. What rendezvous buys is that the receiver's context stays empty, and what it costs is that the receiver does not have the text; one payload size further up, that stops being a preference and becomes the only option.

1 What this run is

property	value
run	`mb-free__transport-rendezvous-2048`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	34
wall time	0.02 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	8	0 eager, 8 rendezvous
tokens sent	21,560	21,560 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

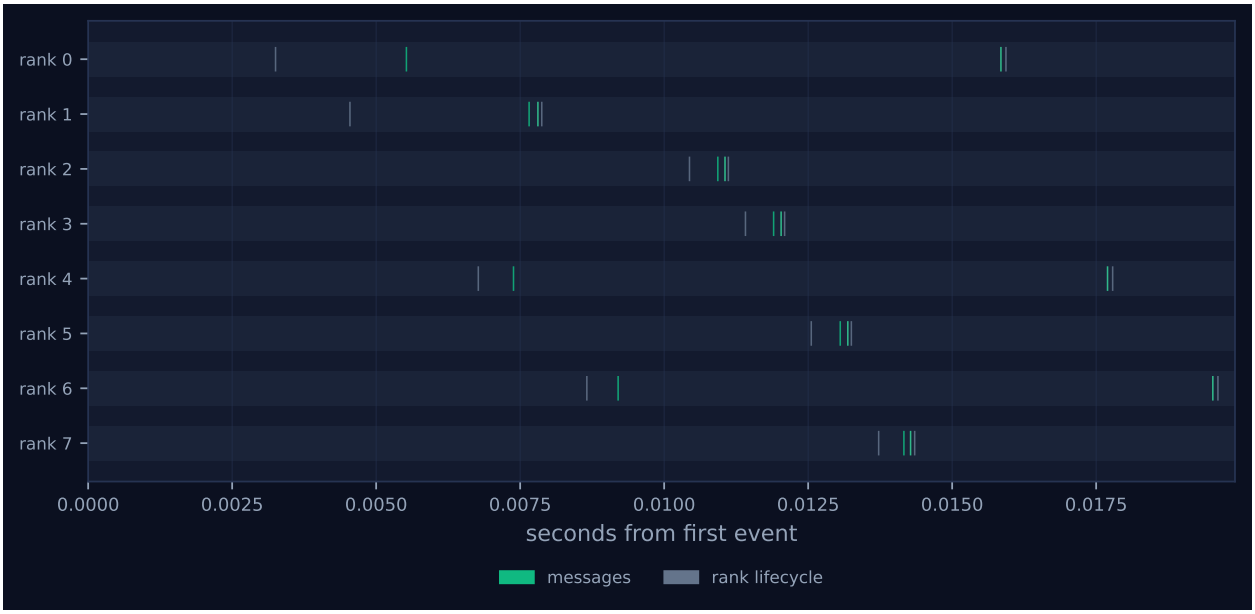

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

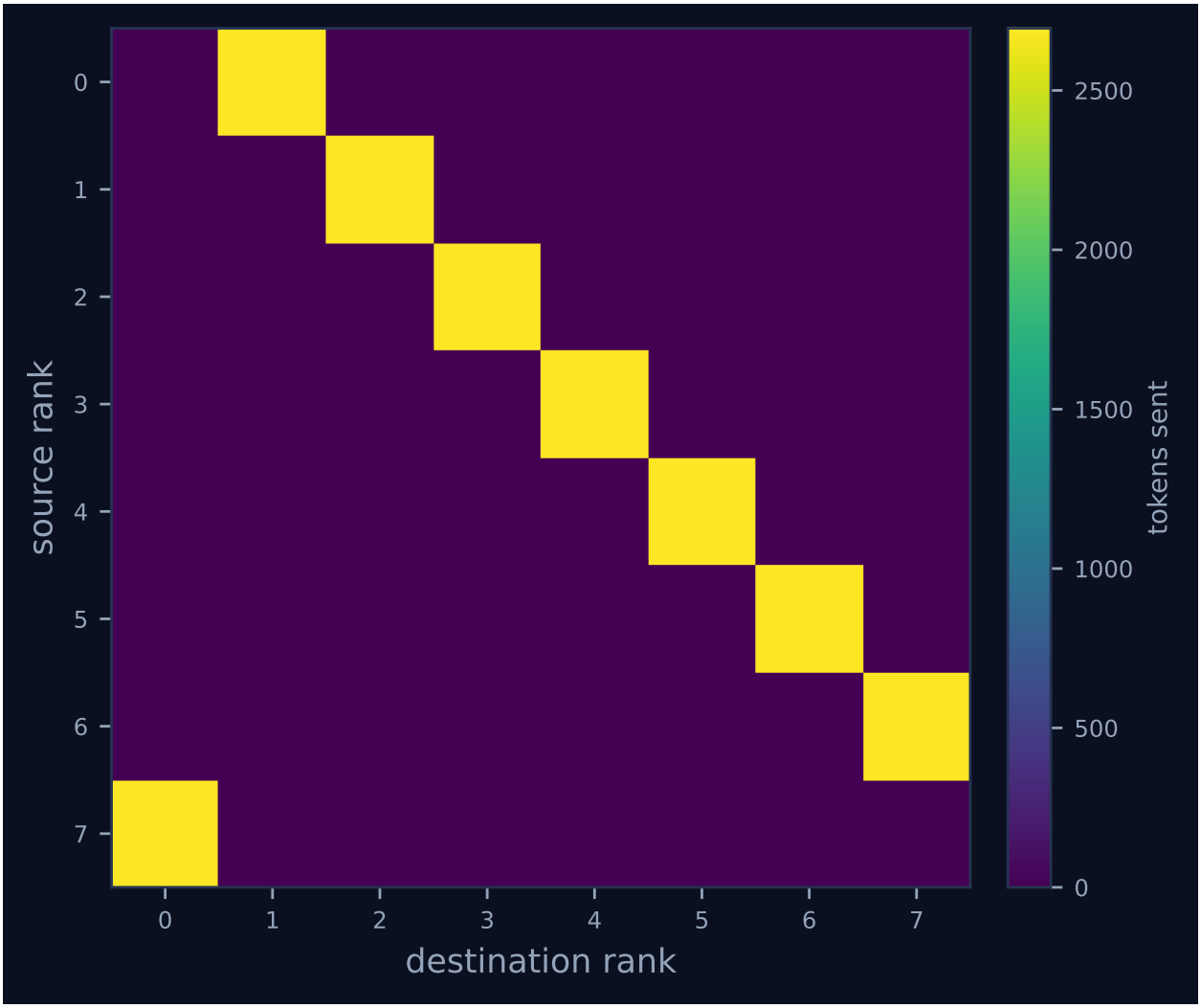

Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	1	1	2,695	0.0	finalized
1	0.00	0.0	0	0	1	1	2,695	0.0	finalized
2	0.00	0.0	0	0	1	1	2,695	0.0	finalized
3	0.00	0.0	0	0	1	1	2,695	0.0	finalized
4	0.00	0.0	0	0	1	1	2,695	0.0	finalized
5	0.00	0.0	0	0	1	1	2,695	0.0	finalized
6	0.00	0.0	0	0	1	1	2,695	0.0	finalized
7	0.00	0.0	0	0	1	1	2,695	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 shows eight lanes with two grey lifecycle diamonds and two green message ticks each and no bars at all, which is a fact about the benchmark rather than about the run: the executor is `none` on all eight ranks, there were zero agent calls, and no `task.*` event was ever emitted. The headline table's "idle fraction 100.0%" and "achieved parallelism 0.00×" consequently record that no rank held a task, not that ranks waited on each other. Ranks arrive over a 10 ms window inside a 20 ms span — this is the shortest run in the sweep, and half of it is rank registration against one SQLite writer. The ring itself is unremarkable: median receive latency 5 ms, maximum 7 ms, every rank finalising within a millisecond of its own receive. Figure 2 carries the structure, one off-diagonal band of 2,695 tokens per edge from rank r to rank $(r{+}1)$ mod 8.

The single row that distinguishes this cell from `mb-free__transport-eager-2048` is context occupancy: 0% on all eight ranks here, 8% there — the largest occupancy anywhere in this sweep, since the two eager cells above it admitted nothing because they sent nothing. Every `msg.recv` in this log reports `admitted: false`, `admitted_tokens: 0` against `tokens: 2695`, and every `rank.finalize` reports `admissions: 0`, `n_items: 0`, `high_water: 0`. The viewer's stats row reads `TOKENS DEFERRED 21.6k`. It is worth noticing what is identical rather than different: both runs wrote the same 10,240-byte payload to their content stores, both deduplicated it to a single blob across all eight ranks, and both spanned about 20–35 ms. Rendezvous did not move less data. It moved the same data and told the receiver less about it.

7 Interpretation

By construction: the ring, the world size, one send and one receive per rank, and the mode. The forcing is more consequential here than in the smaller rendezvous cells. `rank.init` records

`eager_limit: 1000000000`, disabling `_choose_mode`'s automatic threshold — but that threshold's shipped default, `DEFAULT_EAGER_LIMIT` = 2048 tokens, is below this payload's 2,695, so rendezvous is what AgentMPI would have chosen unprompted at this size. This cell is therefore the first in the sweep that agrees with the library's own default, and its eager twin the first that overrides it. That the override succeeded shows the default is conservative: the resource that actually enforces anything is the destination's 4,096-token `unexpected_limit`, and 2,695 clears it with 1,401 to spare.

The deferral arithmetic completes the line. The payload is 2,048 repetitions of `"word "`, 10,240 characters, estimated at $\lceil 10240/3.8 \rceil = 2{,}695$ tokens; with eight ranks and one send each the log-derived total in `metrics.json` is $8 \times 2{,}695 = 21{,}560$, exactly equal to `tokens sent` because every message is rendezvous. That identity is the invariant the field is meant to satisfy, and the results file breaks it: it records 43,120 for this cell, and the per-rank events show `tokens_deferred: 5390` against `tokens_sent: 2695` — a rank deferring twice what it transmitted. Until commit `0fea51d2`, `RankCost.tokens_deferred` accumulated both on rendezvous sends and on every non-admitted receive, so each rank charged its own send once and its predecessor's arrival again. The fix separated the two meanings into `tokens_deferred` (send-side, rendezvous only, necessarily a subset of `tokens_sent`) and `tokens_unadmitted` (receive-side, any transport), under which this run would report 21,560 of each. Because `cost.summarise` reconstructs the figure from the log and was always correct, `metrics.json` and the viewer already show 21,560; the archived results do not. The fix's summary reasons that the paper's transport table is safe because the microbench admits its receives. That is right for the eager rows, which correctly read zero deferred, and wrong for the rendezvous rows: `bench_transport` never passes `admit`, and `Communicator._deliver` defaults it to `False` for rendezvous. All five rendezvous entries in that table — 2,688, 10,784, 43,120, 172,464, 689,856 — are exactly 2× the send-side deferral.

What the cell contributes to the design is the boundary. Below it, at 128 and 512 words, rendezvous is a defensible choice against an eager cost of 0.5% and 2.1% of budget. Here the eager cost has reached 8.2% of the context budget and two-thirds of the credit, and rendezvous starts to look like the sensible default rather than a precaution — which is precisely where AgentMPI's own eager limit sits. One size further up the argument stops being about preference: the 8,192-word eager cell prices at 10,779 tokens, exceeds the credit by 2.63×, and is refused by `_await_eager_credit` before a message row is written, producing 18 events and an empty `messages` table, while the rendezvous cell at the same size runs normally. This run is the last data point at which the sweep can say "both work"; everything above it is the rendezvous column standing alone. The generated finding list is empty and that is correct.

8 Threats to this reading

The zero-occupancy result is the cost of not reading. No `fetch` is called anywhere in this sweep, so the receivers finished holding a digest, a token count and a synopsis; a harness that materialised the payload with `admit=True` would consume the same 2,695 tokens as the eager twin, and the sweep contains no cell that measures partial materialisation through a `View`, which is the case rendezvous exists to make cheap. Second, with `executor: none` and zero agent calls, every context number is an accounting entry in `ContextAccount` rather than a measured prompt; nothing here bears on agent behaviour. Third, the token counts are estimates (`tokenizer_exact: false`) from a character heuristic applied to a payload of one repeated word — a real BPE tokeniser would price this text near 2,048 tokens, putting the eager twin at 50% rather than 66% of credit and rescaling every deferred figure here by about 0.76. Fourth, the wall time is one sample: 20 ms, half of it

rank start-up, so the observation that this cell is nominally the fastest in the sweep is meaningless. Finally, the doubled-deferral diagnosis is reconstructed from per-rank arithmetic and the pre-fix source rather than observed; the code that produced these traces is no longer in the tree, and the argument rests on there being no other way for a rank that made one 2,695-token send and one non-admitted receive to reach 5,390.